

LABORATORIUM: Technologie Chmurowe

===== Z A D A N I E 1 =====

ZASADY REALIZACJI ZADANIA

Zadanie podzielone jest na dwie części : obowiązkową i dodatkową (nieobowiązkową).

1. Realizacja części obowiązkowej odbędzie się na lab 8 (II część lab) oraz na lab 9 (całe lab). Rozwiązanie części obowiązkowej wykonane i przedstawione do oceny w trakcie laboratorium 9 będzie mogło wziąć udział w konkursie na minimalną wielkość obrazu. Rozwiązanie części obowiązkowej, które nie zostaną wykonane w trakcie zajęć należy **OBOWIAZKOWO** opracować i przesłać do dedykowanego katalogu na moodle w terminie podanym przez prowadzącego.
2. Ewentualne sprawozdanie z realizacji części nieobowiązkowej należy tradycyjnie przesłać do dedykowanego katalogu na moodle w terminie podanym przez prowadzącego.

----- CZĘŚĆ OBOWIAZKOWA -----

1. (max. 30%)

Proszę opracować aplikację (dowolny język programowania), która realizować będzie następującą funkcjonalność:

- a. po uruchomieniu kontenera z tą aplikacją, pozostawi ona w logach informację o dacie uruchomienia, imieniu i nazwisku autora programu oraz porcie TCP, na którym aplikacja ta „nasłuchuje”.
- b. aplikacja pozwala na wybranie kraju i miasta (np. z predefiniowanej listy) oraz zatwierdzenie tego wyboru. Na podstawie tych informacji aplikacja powinna wyświetlić aktualną pogodę w wybranej lokalizacji. Zakres informacji pogodowych pozostawiam do decyzji autorów kodu tak jak i projekt i formę UI.

W sprawozdaniu proszę umieścić kod oprogramowania wraz z niezbędnymi komentarzami.

2. (max. 50%)

Opracować plik Dockerfile, który pozwoli na zbudowanie obrazu umożliwiającego uruchomienie aplikacji opracowanej w punkcie 1 jako kontener Docker. Przy ocenie brane będzie sposób opracowania tego pliku (wieloetapowe budowanie obrazu, ewentualne wykorzystanie warstwy scratch, optymalizacja funkcjonowania cache-a w procesie budowania, optymalizacja pod kątem zawartości i ilości warstw, healthcheck itd). Plik Dockerfile powinien również zawierać informację na temat autora tego pliku (imię oraz nazwisko studenta) zgodną ze standardem OCI.

W sprawozdaniu proszę umieścić plik Dockerfile wraz z niezbędnymi komentarzami.

3. (max. 20%)

Należy podać polecenia niezbędne do:

- a. zbudowania opracowanego obrazu kontenera,
- b. uruchomienia kontenera na podstawie zbudowanego obrazu,
- c. sposobu uzyskania informacji z logów, które wygenerowała opracowana aplikacja podczas uruchamiania kontenera (patrz: punkt 1a),
- d. sprawdzenia, ile warstw posiada zbudowany obraz oraz jaki jest rozmiar obrazu.

W sprawozdaniu należy podać treść poleceń (punkty a – d) w raz w ewentualnymi komentarzami oraz zrzut ekranu okna przeglądarki, potwierdzający poprawne działanie systemu.

UWAGA – w przypadku oddawania sprawozdania za pośrednictwem moodle: Wszystkie informacje, które trzeba dostarczyć w sprawozdaniu (punkty 1-4) należy opracować w postaci pliku *zadanie1.md* a ten plik umieścić na koncie GitHub jako zwyczajowy opis zawartości repozytorium git. Na tym repozytorium proszę umieścić też opracowane źródła dla serwera oraz przygotowany plik Dockerfile (oraz wszystkie inne, niezbędne Państwa zdaniem pliki). ***Jako sprawozdanie należy przekazać wyłącznie plik tekstowy zawierający linki do użytego repozytorium na GitHub oraz DockerHub .***

!!!!!!!!!!!! Punkty dodatkowe - KONKURS !!!!!!!!!!!!!

Wśród wszystkich opracowań Zadania 1, część obowiązkowa, które zostaną wykonane w trakcie laboratorium 9 wybrane zostaną (w każdej grupie laboratoryjnej) 3 rozwiązania wykorzystujące obraz o najmniejszym rozmiarze. Autorzy zostaną nagrodzeni punktami zgodnie z wagą dla części nieobowiązkowej zadania, odpowiednio: 50%, 30% oraz 10%.

----- CZĘŚĆ NIEOBOWIĄZKOWA (DODATKOWA) -----

PODSTAWOWE WYMAGANIA przed realizacją części nieobowiązkowej:

1. Aby otrzymać punkty za tą część, należy wykonać **WSZYSTKIE** punkty z części obowiązkowej zadania 1 – w trakcie laboratorium lub w formie sprawozdania przesłanego na moodle.
2. Należy przedstawić rozwiązanie TYLKO JEDNEGO z punktów poniżej (każdy punkt jest rozszerzenie kolejnego czyli zawiera zakres poprzedniego). Punktacja za rozwiązanie **NIE JEST** sumowana – proszę samodzielnie wybrać poziom trudności (wersja od 1 do 3)
3. **(max. +20%)** Każdy opracowany obraz **MUSI** zostać sprawdzony pod względem podatności na zagrożenia. Obraz **NIE POWINIEN zawierać ŻADNYCH** zagrożeń zakwalifikowanych jako **CRITICAL lub HIGH**. Jeśli takie zagrożenia będą znajdować się w obrazie, należy przedstawić uzasadnienie dlaczego występujące zagrożenia można zignorować. Analizę podatności na zagrożenia należy wykonać w oparciu o Docker Scout lub Trivy. Formę raportu świadczącego o braku w/w zagrożeń proszę wybrać samodzielnie. Narzędzie i zasady analizy CVE były przedstawiane na wykładzie 8 i znajdują się w udostępnionych materiałach wykładowych.

UWAGA:

Obrazy niespełniające warunku nr 3 (brak analizy CVE) lub nieposiadające uzasadnienia możliwości zignorowania zagrożeń otrzymają **MNIEJSZĄ LICZBĘ PUNKTÓW** (max. -20%) w stosunku do oceny wynikającej z zrealizowanej wersji części nieobowiązkowej.

1. (max. +20%)

Zbudować obraz kontenera zgodny z OCI zawierający aplikację opracowaną w części obowiązkowej zadania (punkt nr 1), który będzie przeznaczony na platformy sprzętowe: `linux/arm64` oraz `linux/amd64`. W procesie budowy należy wykorzystać builder oparty na sterowniku `docker-container`. Należy potwierdzić, że manifest zawiera deklaracje dla w/w dwóch platform sprzętowych.

2. (max. +50%)

Zbudować obraz kontenera zgodny z OCI zawierający aplikację opracowaną w części obowiązkowej zadania (punkt nr 1), który będzie przeznaczony na platformy sprzętowe: `linux/arm64` oraz `linux/amd64`. W procesie budowy należy wykorzystać builder oparty na sterowniku `docker-container`. Ponadto należy uwzględnić wykorzystanie danych cache dla procesu budowania obrazu (wykorzystanie eksportera `registry` oraz backend-u `inline`). Należy potwierdzić, że manifest zawiera deklaracje dla w/w dwóch platform sprzętowych oraz że dane cache są poprawnie wykorzystywane w procesie budowy obrazu.

3. (max. +80%)

Zbudować obrazy kontenera zgodny z OCI zawierający aplikację opracowaną w części obowiązkowej zadania (punkt nr 1), który będzie przeznaczony na platformy sprzętowe: `linux/arm64` oraz `linux/amd64`. W procesie budowy należy wykorzystać builder oparty na sterowniku `docker-container`. W pliku `Dockerfile` należy wykorzystać rozszerzony frontend `buildkit`, który umożliwi bezpośrednie wykorzystanie kodów aplikacji umieszczonych w swoim repozytorium publicznym na GitHub w procesie budowania obrazów (funkcjonalność `mount ssh / mount secret`). W procesie budowania obrazu należy wykorzystywać dane cache dla procesu budowania obrazu (wykorzystanie eksportera `registry` oraz backend-u `registry` w trybie `max`). Należy potwierdzić, że manifest zawiera deklaracje dla w/w dwóch platform sprzętowych oraz że został utworzony i jest poprawnie wykorzystywany obraz z danymi cache dla procesu budowy obrazu.

SPRAWOZDANIE:

Opracowane obrazy należy przesłać do swojego repozytorium na DockerHub. W sprawozdaniu należy podać wykorzystane instrukcje wraz z wynikiem ich działania oraz ewentualnymi komentarzami. W przypadku podpunktu 3 proszę nie przekazywać danych wrażliwych (jeśli takie będą wykorzystywane) a jedynie przedstawić wynik wykonywanych działań (poleceń).

Ponownie wszystkie informacje, które trzeba dostarczyć w postaci pliku `zadanie1_dod.md` a ten plik umieścić na koncie GitHub jako zwyczajowy opis zawartości repozytorium git. Na tym repozytorium proszę umieścić też opracowane źródła dla serwera oraz przygotowany plik `Dockerfile` (oraz wszystkie inne, niezbędne Państwa zdaniem pliki). ***Jako sprawozdanie należy przekazać wyłącznie plik tekstowy zawierający linki do użytego repozytorium na GitHub oraz DockerHub.***

===== UWAGI KOŃCOWE: =====

1. Zadanie należy wykonać SAMODZIELNIE. W przypadku kopii (tak typu **Ctrl C – Ctrl V**) lub modyfikacji nieistotnych merytorycznie) ocena z zadania zostanie **PODZIELONA** przez liczbę „współtwórców”.
2. Oceniający zastrzega sobie prawo do podwyższenia oceny jeśli dane rozwiązanie chwyci go za serce albo umysł.